UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment 7

Student Name: Pratham **UID: 25MCI10178**

Branch: MCA (AI & ML) **Section/Group: 25MAM_KAR-1_A**

Semester: II **Date of Performance: 24/02/26**

Subject Name: Technical Training **Subject Code: 25CAP-652**

Experiment Aim: To understand and implement different types of SQL joins (INNER, LEFT, RIGHT, SELF, CROSS) in PostgreSQL for retrieving meaningful relationships between students, courses, and departments.

Objectives:

- Demonstrate how **INNER JOIN** retrieves common records between tables.
- Use **LEFT JOIN** to identify students without course enrollment.
- Apply **RIGHT JOIN** to display all courses, even those without students.
- Explore **SELF JOIN / multiple joins** to connect students with their department information.
- Implement **CROSS JOIN** to generate all possible student-course combinations.

Experiment Steps:

1. Write queries to list students with their enrolled courses (INNER JOIN).

```
SELECT s.student_id, s.student_name, c.course_name
FROM students s
INNER JOIN enrollments e ON s.student_id = e.student_id
INNER JOIN courses c ON e.course_id = c.course_id;
```

	student_id integer	student_name character varying (100)	course_name character varying (100)
1	1	Alice	Database Systems
2	2	Bob	Database Systems
3	2	Bob	Operating Systems
4	3	Charlie	Thermodynamics
5	4	David	Circuit Analysis

2. Find students not enrolled in any course (LEFT JOIN).

```
SELECT s.student_id, s.student_name

FROM students s

LEFT JOIN enrollments e ON s.student_id = e.student_id

WHERE e.course_id IS NULL;
```

	student_id [PK] integer	student_name character varying (100)
1	5	Eva
2	6	Frank

3. Display all courses with or without enrolled students (RIGHT JOIN).

```
SELECT c.course_id, c.course_name, s.student_name

FROM students s

RIGHT JOIN enrollments e ON s.student_id = e.student_id

RIGHT JOIN courses c ON e.course_id = c.course_id;
```

	course_id integer	course_name character varying (100)	student_name character varying (100)
1	1	Database Systems	Alice
2	1	Database Systems	Bob
3	2	Operating Systems	Bob
4	3	Thermodynamics	Charlie
5	4	Circuit Analysis	David
6	5	Marketing 101	[null]

4. Show students with department info using SELF JOIN or multiple joins.

```
SELECT s.student_id, s.student_name, d.department_name

FROM students s

INNER JOIN departments d ON s.department_id = d.department_id;
```

	student_id integer	student_name character varying (100)	department_name character varying (100)
1	1	Alice	Computer Science
2	2	Bob	Computer Science
3	3	Charlie	Mechanical Engineering
4	4	David	Electrical Engineering
5	5	Eva	Business Administration
6	6	Frank	Computer Science

5. **Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)**

```
SELECT s.student_name, c.course_name

FROM students s

CROSS JOIN courses c;
```

student_name	course_name
Alice	Database Systems
Bob	Database Systems
Charlie	Database Systems
David	Database Systems
Eva	Database Systems
Frank	Database Systems
Alice	Operating Systems
Bob	Operating Systems
Charlie	Operating Systems
David	Operating Systems
Eva	Operating Systems
Frank	Operating Systems
Alice	Thermodynamics
Bob	Thermodynamics
Charlie	Thermodynamics
David	Thermodynamics
Eva	Thermodynamics
Frank	Thermodynamics
Alice	Circuit Analysis
Bob	Circuit Analysis
Charlie	Circuit Analysis
David	Circuit Analysis
Eva	Circuit Analysis
Frank	Circuit Analysis
Alice	Marketing 101
Bob	Marketing 101
Charlie	Marketing 101
David	Marketing 101
Eva	Marketing 101
Frank	Marketing 101

Outcomes:

- Gain practical knowledge of different SQL joins (INNER, LEFT, RIGHT, SELF, CROSS).
- Learn how to retrieve, combine, and analyze data across multiple tables.
- Develop skills to identify missing or related data using joins.